15 October 2026

The Editor-in-Chief
Computers and Electrical Engineering
Elsevier

Dear Editor,

We submit our manuscript, **"Benchmarking Post-Quantum Key Encapsulation in S3-Compatible Object Storage under Client Concurrency"**, for consideration as a research article in *Computers and Electrical Engineering*. The manuscript is transferred from *Computers & Security* (COSE-D-26-06498), which returned it without review because that journal does not consider papers in which cryptology is a principal topic, and Elsevier's transfer service suggested your journal.

The paper is a systems and performance study, not a cryptographic one. Data encrypted today with classical public-key algorithms could be decrypted once large quantum computers exist, so the client-side encryption that protects cloud object storage must move to post-quantum key encapsulation such as the NIST standard ML-KEM, which adds a key-encapsulation step and a larger ciphertext to every stored object. So far, this cost has been measured for the algorithm on its own and for securing network connections, but not on the data path of a storage system that serves many clients at once. We do not propose or analyse cryptographic algorithms; we measure what deploying the standard costs in a concurrent storage system and why.

We measured per-object ML-KEM-768 against X25519, RSA-4096, a symmetric-only baseline and two hybrid constructions on an S3-compatible object store, with a Python client on x86 and ARM and a native Go reader, at 1 to 500 concurrent workers. The study comprises an exploratory campaign (1,970 runs), a pre-registered confirmatory campaign (1,495 runs, 97.6 million requests) analysed with paired equivalence tests and Holm correction, and pre-registered follow-up controls (480 runs with the Python client and 90 with the native reader). The main findings are:

- ML-KEM lowered throughput only where the client was at or near CPU saturation, with one unexplained exception that we report.
- In the saturated pre-registered cell the Python client was 4.0 % (x86) and 4.9 % (ARM) slower than with X25519. Controls on x86 traced this deficit mainly to the library binding, and with a single foreign call ML-KEM was equivalent to X25519 within ±2 %.
- A native reader saturated on one core was 9.5 % slower, showing that the size of the cost depends on the implementation.
- The cost every implementation pays is storage: 1,056 more bytes of ciphertext per object than X25519.

We believe the work fits *Computers and Electrical Engineering* because it is an experimental evaluation of a computer system: it locates a performance cost in client CPU, in a software binding and in storage overhead, and it gives engineers a measurement procedure (client CPU per request under saturation) to apply before migrating. The paper states its limits openly: two clients, a single-node store and one cloud provider.

We confirm that this manuscript is original, has not been published previously, and is not under consideration for publication elsewhere. A preprint has been submitted to arXiv. All authors have read and approved the manuscript and agree with its submission. In line with the journal's double-anonymised review, the manuscript contains no author information; author details and the link to the public code and data repository are given on the separate title page.

Thank you for considering our submission.

Yours sincerely,

Revanth Reddy Lingala (corresponding author), on behalf of all authors
Department of Information Technology, Chaitanya Bharathi Institute of Technology, Hyderabad, Telangana, India
revanthreddy0906@gmail.com